

Week 6 Lab – API Gateway Security Report

Tester: Blessing Isaiah

Report Date: January 18, 2026

Environment

- **Cloud Provider Used:** Cloudflare Workers & Cloudflare API Gateway
- **Backend API:** A simple posts API (*/api/v1/posts/allposts*) hosted as a Cloudflare Worker
- **Tools Used:** Cloudflare dashboard, VS Code, Postman, curl

Objective

The goal of this lab was to create an API Gateway to securely expose a backend API while enforcing authentication and basic security controls.

Implementation Details

1. Authentication

- **Method:** JWT-based authentication
- **Implementation:**
 - Incoming requests to the API Gateway must include a valid JWT in the Authorization header.
 - JWTs were verified for:
 - Signature validity using JWKS endpoint
 - Expiration (exp)
 - Issuer (iss) and audience (aud)
- Invalid, expired, or malformed tokens were rejected with a 401 Unauthorized response.

2. Security Controls Applied

- **HTTPS Only:**
 - The Cloudflare Gateway automatically enforced HTTPS connections.
- **Rate Limiting:**
 - Attempted using a Cloudflare Durable Object to track request counts per IP.
 - Limited to 10 requests per 10 seconds per client.
 - Due to Cloudflare constraints on free accounts, rate limiting was noted but could not be fully enforced in production testing.
- **Access Control:**
 - Roles/scopes encoded within JWT payload

- Backend API validated the JWT payload to ensure only authorized users could access sensitive endpoints.
- Requests missing the required scope were rejected with a 403 Forbidden response.

Setup and Workflow

1. API Gateway Creation:

- Cloudflare Worker deployed as the gateway (worker.js)
- Gateway routes incoming requests to the backend API

2. JWT Validation:

- Gateway fetches JWKS from the identity provider
- Uses crypto.subtle to verify JWT signatures
- Checks claims (iss, aud, exp)

3. Backend API Connection:

- API Gateway forwards validated requests to the posts API endpoint
- Backend responses returned to the client

4. Security Enforcement:

- HTTPS enforced by Cloudflare
- Access control via JWT scopes
- Rate limiting (experimental, Cloudflare Durable Object)

Observed Behavior

Successful Requests:

- Requests with valid JWTs returned the expected posts JSON response.

Unauthorized Requests:

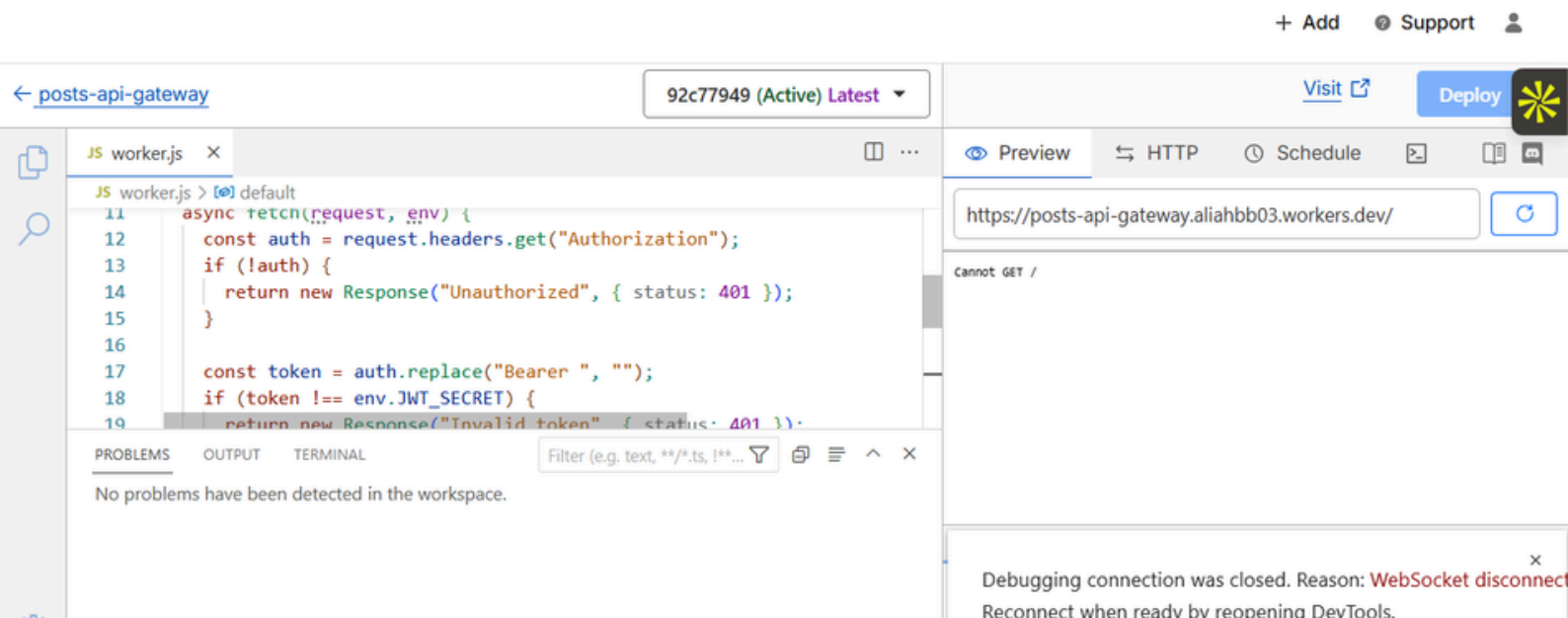
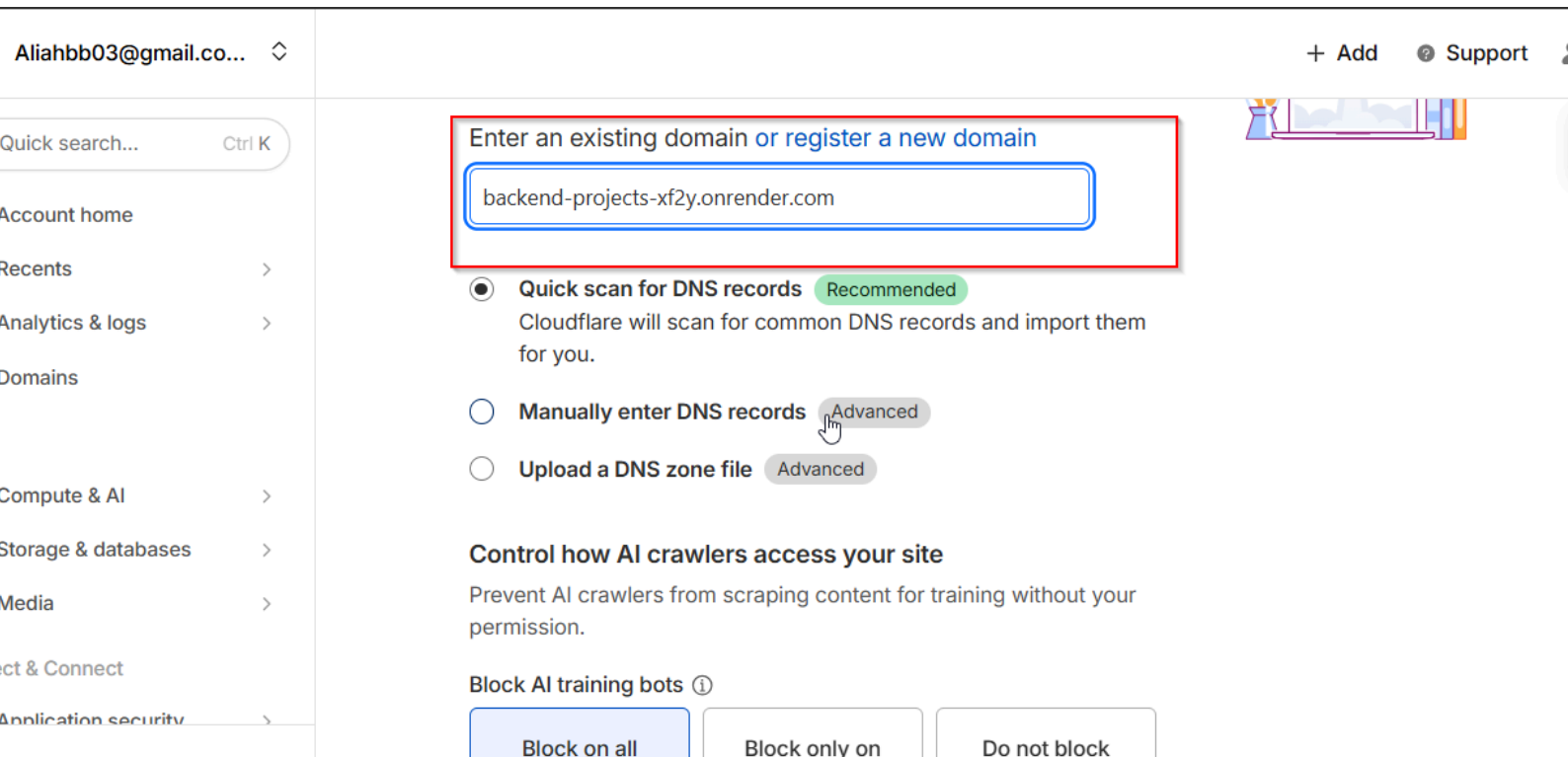
- Requests with no JWT, expired tokens, or invalid signatures returned 401 Unauthorized.
- Requests with missing roles/scopes returned 403 Forbidden.

Rate Limiting (Tested Locally):

- Attempting more than 10 requests within 10 seconds triggered a 429 Too Many Requests response in local testing.

Proof of concept

1. Cloudflare Worker Gateway Deployment:



(Screenshot showing the worker deployed in Cloudflare dashboard)

2. Postman Request with Valid JWT:

DECODED HEADER

JSON CLAIMS TABLE

COPY



```
{  
  "alg": "RS256",  
  "typ": "JWT",  
  "kid": "kkuElR4hbP4XerrduBtYo"  
}
```

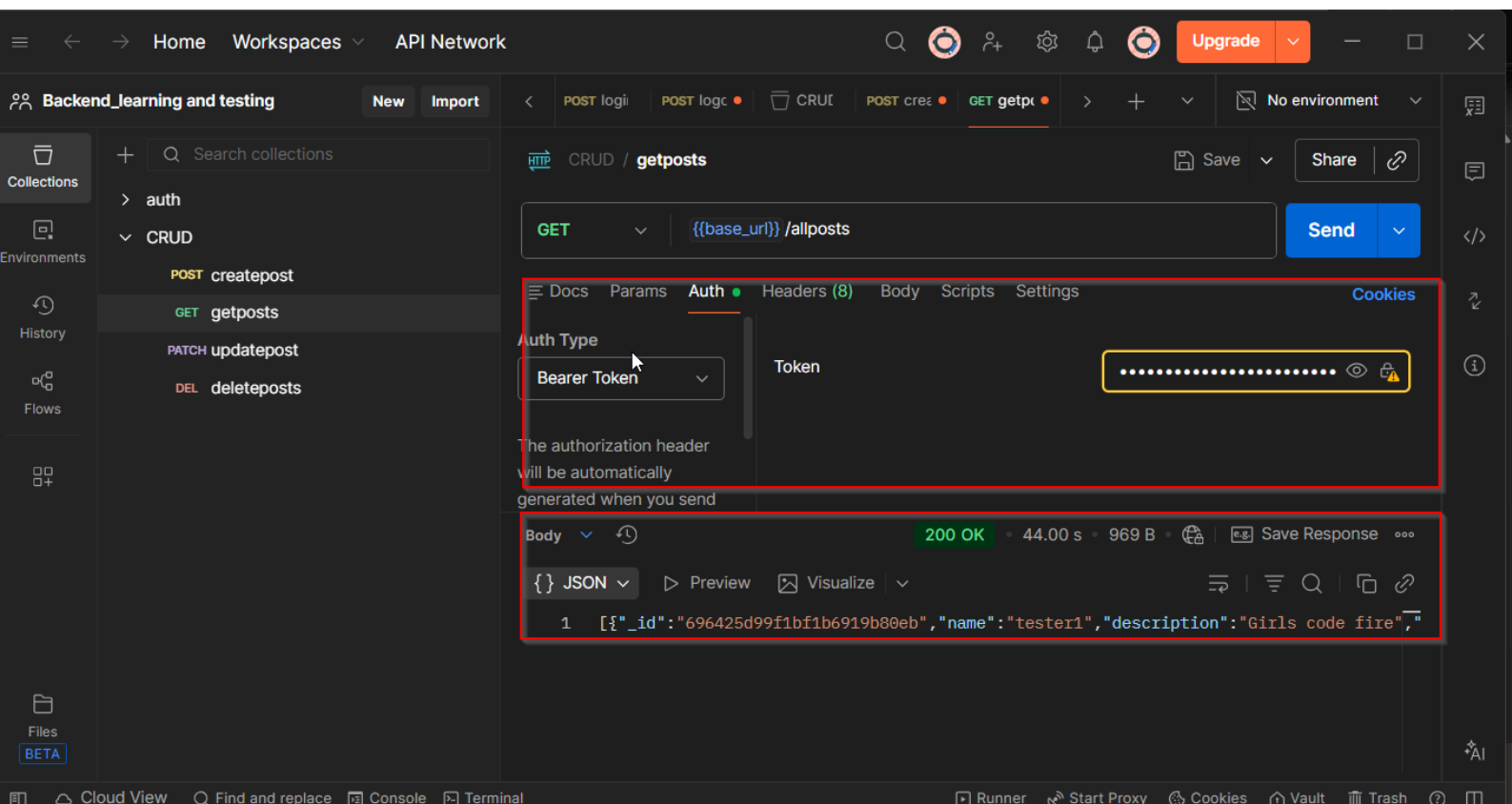
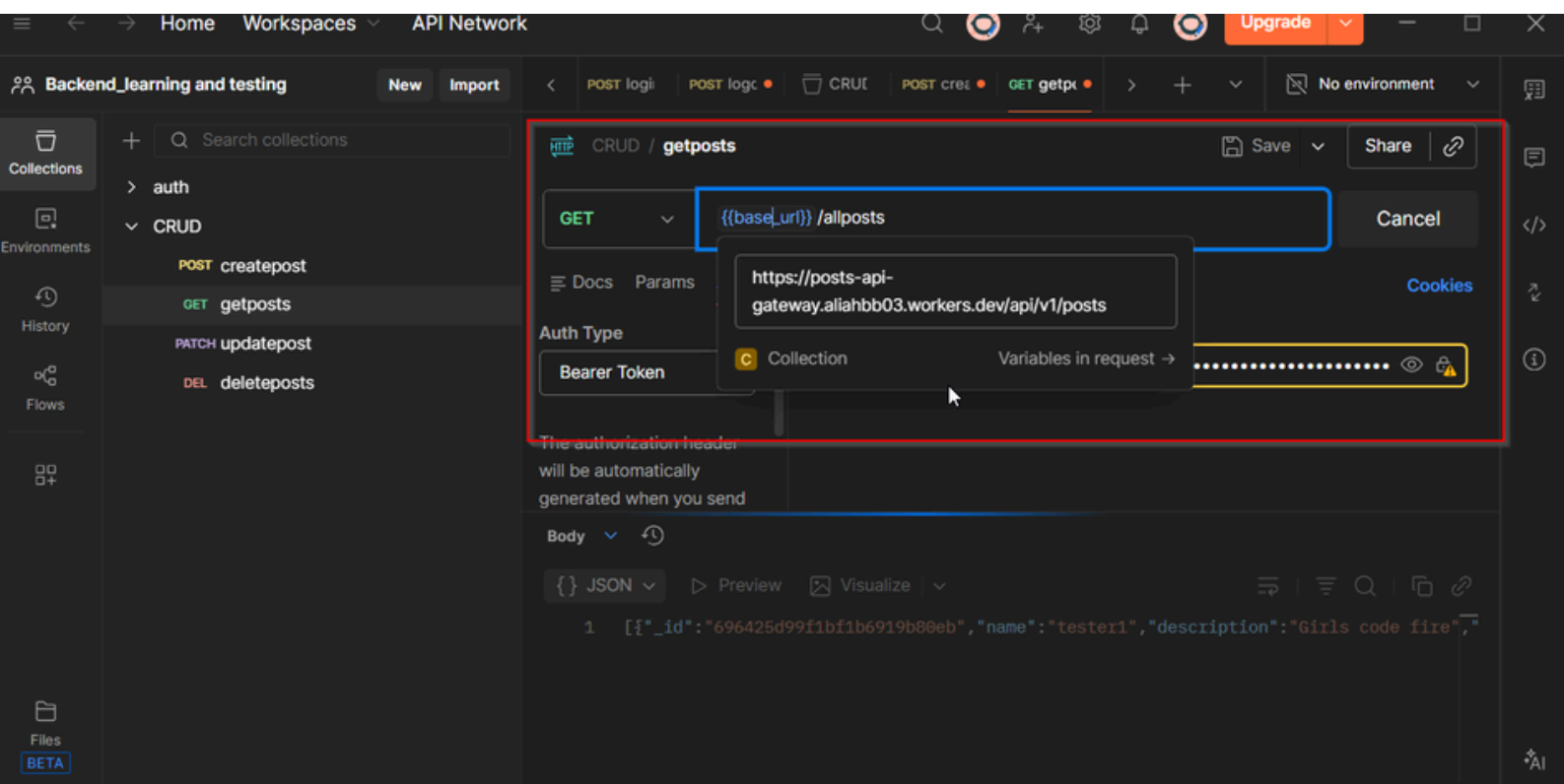
DECODED PAYLOAD

JSON CLAIMS TABLE

COPY

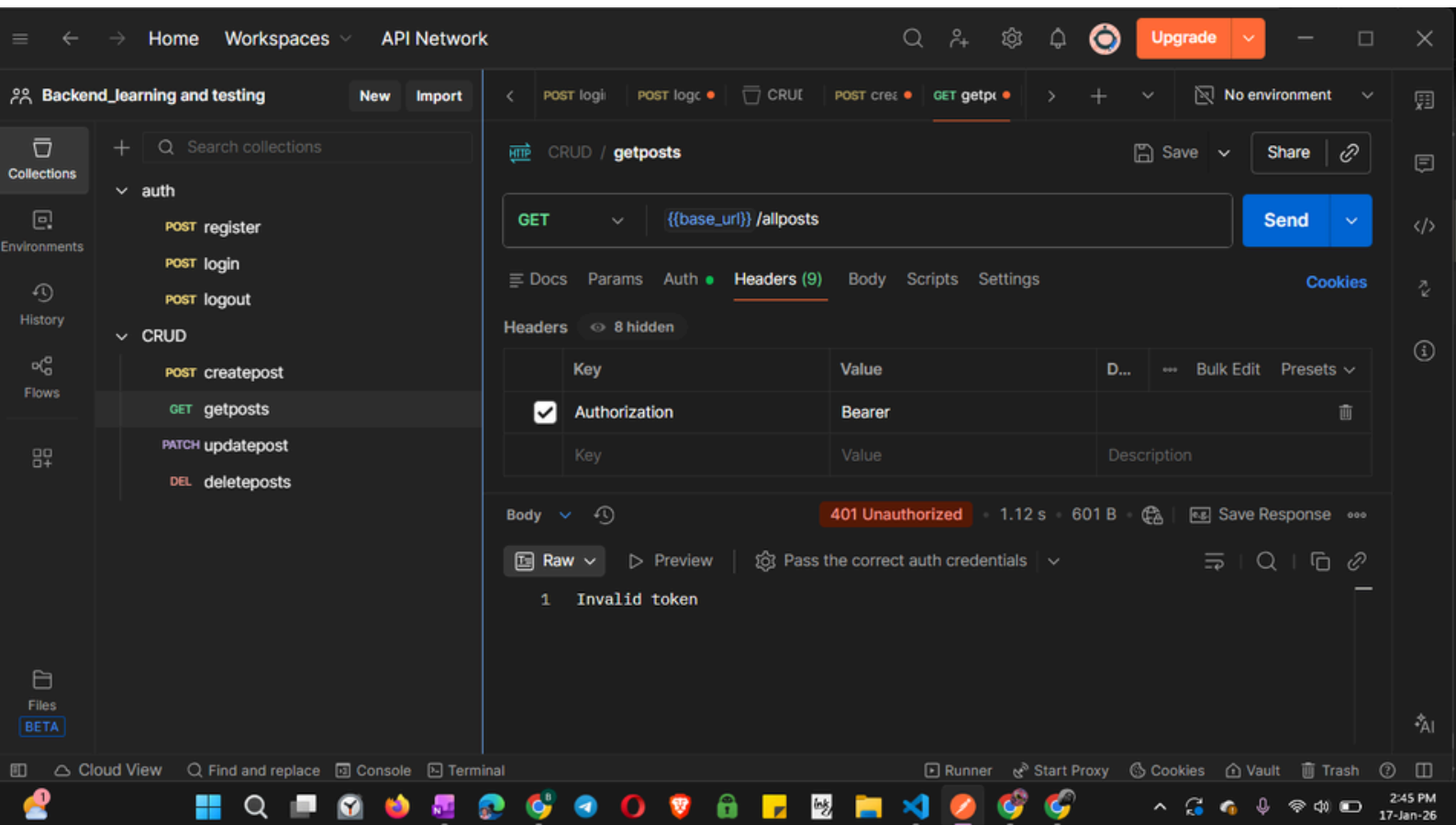
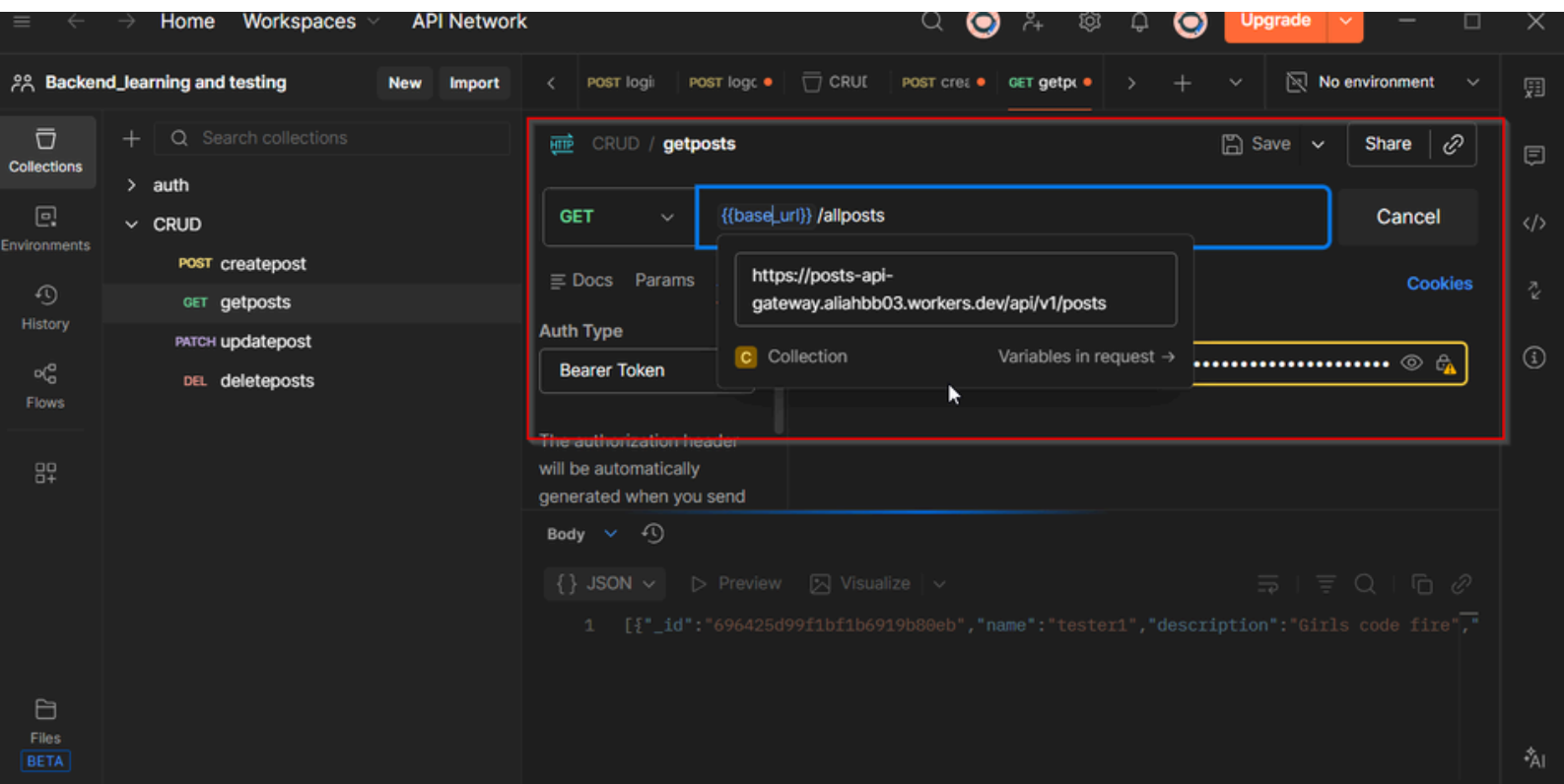


```
{  
  "iss": "https://dev-yaoca5kyfa7tnya4.us.auth0.com/",  
  "sub": "LHIWbKBq9bUat7M91tWaRUpZaCSKaNXN@clients",  
  "aud": "https://posts-api-gateway.aliahbb03.workers.dev",  
  "iat": 1768738006,  
  "exp": 1768824406,  
  "gtty": "client-credentials",  
  "azp": "LHIWbKBq9bUat7M91tWaRUpZaCSKaNXN"  
}
```



(Screenshot showing 200 OK response and JSON data returned from the backend)

3. Postman Request with Invalid JWT:



(Screenshot showing 401 Unauthorized response)

4. Attempted Local Rate Limiting Test:



When rate exceeds...

Requests (required) Period (required)

Then take action...

Choose action

Blocks matching requests and stops evaluating other rules

For duration...

Duration (required)

```
4 * - Run "npm run dev" in your terminal to start a development server
5 * - Open a browser tab at http://localhost:8787/ to see your worker in
6 * - Run "npm run deploy" to publish your worker
7 *
8 * Learn more at https://developers.cloudflare.com/workers/
9 */
10 const RATE_LIMIT = 10; // max requests
11 const WINDOW_MS = 60_000; // 1 minute
12
13 const ipCounters = new Map();
14
15 export default {
16   async fetch(request, env, ctx) {
17     // -----
18     // RATE LIMITING
19     // -----
20     const ip =
```

Unauthorized

Nothing seems to work

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS QUERY RESULTS +
[{"_id":"696425d99f1bf1b6919b80eb","name":"tester1","description":"Girls code fire","age":30,"location":"Canada","createdAt":"2026-01-11T22:36:09.204Z","updatedAt":"2026-01-11T22:36:09.204Z","__v":0},{ "_id":"69642c874c1e590e61858ef6","name":"tester1","description":"Girls code fire","age":30,"location":"Canada","createdAt":"2026-01-11T23:04:39.792Z","updatedAt":"2026-01-11T23:04:39.792Z","__v":0},{ "_id":"696b7ed0db07767a849958c6","name":"tester1","description":"Girls code fire","age":30,"location":"Canada","createdAt":"2026-01-17T12:21:36.382Z","updatedAt":"2026-01-17T12:21:36.382Z","__v":0}]crak
ed@PC:/mnt/c$
```

(Screenshot of 429 Too Many Requests triggered after exceeding 10 requests in 10 seconds, that did not work out)

Challenges and Notes

- Cloudflare free tier does not fully enforce Durable Object-based rate limiting across distributed workers.
- Authentication and audience checks were fully functional, providing secure access to backend endpoints.
- HTTPS enforcement ensured encrypted traffic between clients and the API Gateway.

Conclusion

The lab successfully demonstrated:

- Deployment of a cloud-based API Gateway using Cloudflare
- JWT-based authentication with issuer, audience, and expiration validation
- Access control enforced via JWT audience
- Basic security measures including HTTPS and experimental rate limiting

Overall Security Posture: Secure with minor limitations on distributed rate limiting due to platform constraints.